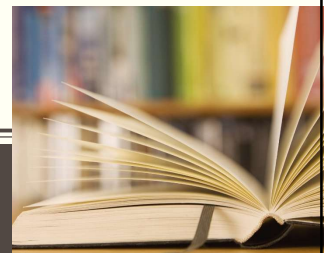
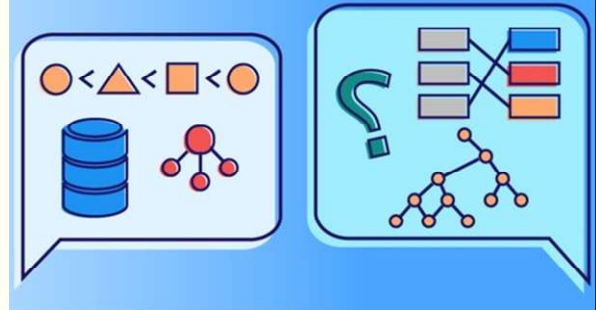


ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 31/08

- Resolución de un ejercicio completo con registros (struct)

2

Contraseña del estudiante

2u6sxq

3

Estructuras de datos: Clasificaciones

- Según donde se almacenan
 - Internas (en memoria principal)
 - Externas (en memoria auxiliar)
- Según tipos de datos de sus componentes
 - Homogéneas (todas del mismo tipo)
 - No homogéneas (pueden ser de distinto tipo)
- Según la implementación
 - Provistas por los lenguajes (básicas)
 - Abstractas (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- Según la forma de almacenamiento
 - Estáticas (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
 - Dinámicas (su tamaño varía durante el módulo y sus posiciones también)

4

Estructuras (Struct o Registros)

- Colección de variables relacionadas, bajo un mismo nombre
- Pueden contener variables de diferentes tipos de datos
- Se usa habitualmente para definir registros que van a ser almacenados en archivos o en arreglos.
- Combinada con punteros, pueden crear listas enlazadas, pilas, colas y árboles.

5

Guia 13 – Ejercicio 2

Crear una estructura empleado que contiene la siguiente información:

- Nombre
- Dirección (calle, número, barrio: “María Selva”, “Candioti”, “Sur”, etc)
- Fecha de nacimiento (día,mes,año)
- Sexo (M = masculino, F = femenino)
- Sueldo
- Categoría (“Operario”, “Jefe de área”, “Gerente”)
- Fecha de ingreso (día,mes,año)

6

Guia 13 – Ejercicio 2

Utilizando la estructura, declarar un vector de 100 elementos de estructuras de ese tipo para administrar los empleados de una empresa.

Luego:

- Imprimir el nombre y el sexo del cuarto empleado.
- Imprimir los nombres de todos los empleados que tienen un salario mayor a \$57500.
- Aumentar un 15% el sueldo a los empleados con antigüedad mayor a 5 años.
- Listar los empleados menores de 25 años.
- Listar los empleados que viven en el barrio Sur.
- Listar los operarios que viven en el barrio Candioti.
- Listar el vector ordenado por nombre.

7

Algoritmos y Estructuras de Datos – AEDD – Asistencia 31/08

Contraseña del estudiante

2u6sxq

8

Estructuras definidas

Se definieron 3 estructuras:

Direccion

Fecha

Empleado

9

Uso de estructuras como parámetros en funciones

Paso de Estructuras a Funciones:

- Paso de una estructura completa
- Paso de campos individuales

Por defecto, los structs se pasan por valor.

Para pasar una estructura por Valor:

- Se pasa una copia de la estructura

Para pasar una estructura por Referencia:

- Se pasa su dirección

10

Algoritmos y Estructuras de Datos – AEDD – Agenda 31/08

✓ Resolución de un ejercicio completo con registro

¡¡Objetivo cumplido!!!

11

AEDD – Otras tareas !!!!!

Capítulo 13 - “Estructuras”

Libro: “C++ para ingeniería y ciencias” 2da edición - Gary J.
Bronson, pág. 711

Capítulo 6 - “Tipos Compuestos”

Libro: Benjumea-Roldan, pág. 64
Universidad de Málaga

“Estructuras” - Página 543

Libro: “Fundamentos de la programación” - Luis Hernández
Yáñez - Universidad Complutense

12

Algoritmos y Estructuras de Datos – AEDD – Agenda 31/08

- Definición de Estructura.
- Definición de dato TDA.
- Tipo de dato: Fracción.
- Implementación y uso de TDA.
- Definición y uso de TDA.
- Ejemplo: Punto en el plano.
- Ejemplo: Recta en el plano.
- Rol de la abstracción.
- Ventajas de los TDA

13

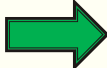
Estructuras de datos: Clasificaciones

- Según donde se almacenan
 - Internas (en memoria principal)
 - Externas (en memoria auxiliar)
- Según tipos de datos de sus componentes
 - Homogéneas (todas del mismo tipo)
 - No homogéneas (pueden ser de distinto tipo)
- Según la implementación
 - Provistas por los lenguajes (básicas)
 - Abstractas (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- Según la forma de almacenamiento
 - Estáticas (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
 - Dinámicas (su tamaño varía durante el módulo y sus posiciones también)

14

Estructuras de Datos: Clasificaciones

Según la implementación

- Provistas por los lenguajes: Básicas 
- Abstractas: TDA - Tipo de dato abstracto que puede implementarse de diferentes formas.

Array
Struct
String
File



Un TDA es un tipo de dato definido por el programador que se puede manipular de un modo similar a los tipos de datos provistos por el lenguaje.

Un TDA está formado por un conjunto de valores válidos de datos y un conjunto de operaciones primitivas que se pueden realizar sobre esos valores.

Los usuarios pueden crear variables con valores del conjunto válido y operar sobre esos valores.

15

Definición de TDA

Los TDA proporcionan un mecanismo adicional mediante el cual se realiza una separación clara entre la interfaz y la implementación del tipo de dato.

La implementación de un TDA consta de:

- La representación: elección de las estructuras de datos, que determina cómo se almacenan internamente los datos.
- Las operaciones: elección de los algoritmos, define cómo se manipulan esos datos.

La interfaz del TDA se asocia con las operaciones y las estructuras de datos del TDA, y es visible al exterior. Esto significa que los usuarios del TDA interactúan con él a través de una serie de operaciones predefinidas, sin necesidad de conocer los detalles de su implementación.

16

Revisando algunos tipos de datos básicos

En C++, el tipo `int` (enteros) se corresponde con el conjunto de valores del siguiente rango: $-2.147.483.648$ a $+2.147.483.647$; y las operaciones primitivas posibles son:

- Operadores de asignación: `=`, `*=`, `/=`, `%=`, `+=`, `-=`
- Operadores aritméticos: `+(signo)`, `-(signo)`, `*`, `/`, `%`, `+`, `-`
- Operadores de incremento y decremento: `++`, `--`
- Operadores relacionales: `==`, `!=`, `>`, `<`, `>=`, `<=`

Similarmente podemos emplear el tipo `float` (tipo de coma flotante) para manipular números decimales (reales).

- Con su conjunto de valores y operaciones permitidas.

17

Tipo de dato: Fracción

En C++ no está predefinido el tipo de datos fracción.

Si necesitamos almacenar y operar con fracciones, deberemos definir un tipo de dato:

- nuevo
- diseñado y definido por el programador
- que no preexiste (será abstracto),
- creado con los datos y operaciones primitivas que provee el sistema.

18

Tipo de dato: Fracción

- Una forma de definirlo es pensar en representar a una fracción como una estructura (struct) compuesta de dos campos de tipo int: numerador y denominador.
- Una estructura permite que un grupo de variables que tienen una cierta relación lógica sean tratadas como un TODO.
- Podrán declararse variables o funciones del nuevo tipo definido.

```
struct fraccion {
    int num;
    int den;
};
fraccion fra1, fra2, fra3;
```

19

Tipo de dato: Fracción

- Con esta definición *fracción* se corresponde con el conjunto de valores formado por todos los pares ordenados de números enteros:

Rango: -2.147.483.648 a +2.147.483.647

- También debe definirse un conjunto de operaciones primitivas (interfaz) que se pueden realizar sobre datos del tipo fracción.
- Para describir un TDA, y en particular la interfaz (operaciones permitidas), se puede usar una notación informal o una especificación formal. Optamos por la primera, indicando:

Operación (tipos de argumentos) $\Rightarrow$ resultado, descripción

20

Tipo de dato: Fracción – Operaciones permitidas

Para el TDA fraccion, el conjunto de operaciones primitivas (interfaz) puede ser:

- **fracIguales (fraccion, fraccion) → boolean;** determina si dos fracciones son iguales, esto es si tienen el mismo numerador y denominador.
- **fracEquiv (fraccion, fraccion) → boolean;** determina si dos fracciones son equivalentes, es decir si tienen el mismo valor.
- **asigFrac (entero, entero) → fraccion;** devuelve una fracción que tiene como numerador al 1er. argumento y como denominador al 2do.
- **sumFrac (fraccion, fraccion) → fraccion;** realiza la operación suma de dos fracciones.
- **multFrac (fraccion, fraccion) → fraccion;** realiza la operación multiplicación de dos fracciones.
- **restFrac (fraccion, fraccion) → fraccion;** realiza la operación resta de dos fracciones.
- **divFrac (fraccion, fraccion) → fraccion;** realiza la operación división de dos fracciones.
- **printFrac (fraccion) void;** realiza la impresión de una fracción con el formato literal "numerador / denominador".

21

Tipo de dato: Fracción

Luego de definido el tipo de dato abstracto (valores y operaciones) y teniendo declaradas las variables fa, fb y fc, un módulo (código de usuario) que emplee este nuevo tipo de datos sería, por ejemplo:

```
#include <iostream>
using namespace std;
#include "fraccion.h"

int main (void) {
    int n, d;
    fraccion fa, fb,fc;
    cout << "Ingrese una fraccion como num seguido de den: ";
    cin >> n >> d;
    fa = asigFrac (n,d);
    fb = asigFrac (3,5);
    fc = multFrac (fa,fb);
    cout << endl << "El resultado de la multiplicacion por 3/5 es: ";
    printFrac (fc);
    system("pause");

    return 0;
}
```

22

Tipo de dato: Fracción

Así por ejemplo la declaración (implementación) de las funciones `asigFrac`, `fracEquiv`, `multFrac` y `sumFrac`, podría ser:

/ Devuelve una fracción */*

```
fraccion asigFrac (int x,int y)
{
    fraccion aux;
    aux.num = x;
    aux.den = y;
    return aux;
}
```

/ Multiplica dos fracciones */*

```
fraccion multFrac (fraccion a, fraccion b)
{
    fraccion aux;
    aux.num = a.num * b.num;
    aux.den = a.den * b.den;
    return aux;
}
```

/ Suma dos fracciones */*

```
fraccion sumFrac (fraccion a,fraccion b)
{ fraccion aux;
  aux.den = a.den * b.den;
  aux.num = b.den * a.num +
            a.den * b.num;
  return aux;
}
```

/ Determina si dos fracciones son equivalentes */*

```
int fracEquiv (fraccion a, fraccion b)
{
    return (a.num * b.den == a.den * b.num);
}
```

23

Tipo de dato: Fracción

La función definida para sumar devuelve una fracción que no está reducida (simplificada). Si se tiene definida una función que calcule el mínimo común múltiplo entre 2 enteros, la función ***sumFrac*** podría definirse como:

```
fraccion sumFrac (fraccion a, fraccion b) {
    fraccion aux;
    aux.den = mcm (a.den, b.den);
    aux.num = (aux.den / a.den) * a.num + ( aux.den/ b.den) * b.num;
    return aux;
}
```

24

Tipo de dato: Fracción

Tener en cuenta que:

El usuario (programador) define:

- cómo va a representar el nuevo tipo de datos que está creando (TDA),
- cuáles van a ser las operaciones primitivas (interfaz) y
- el código (implementación) de dichas operaciones.

El programador que utilice el TDA (cliente) debe conocer la representación y la interfaz.

La implementación queda oculta para el programador “cliente”.

25

Implementación de TDA en C++

Una de las características de C++ que permite implementar TDA son los archivos de inclusión o cabecera que se utilizan para agrupar en ellos variables externas, declaraciones de datos comunes y prototipos de funciones.

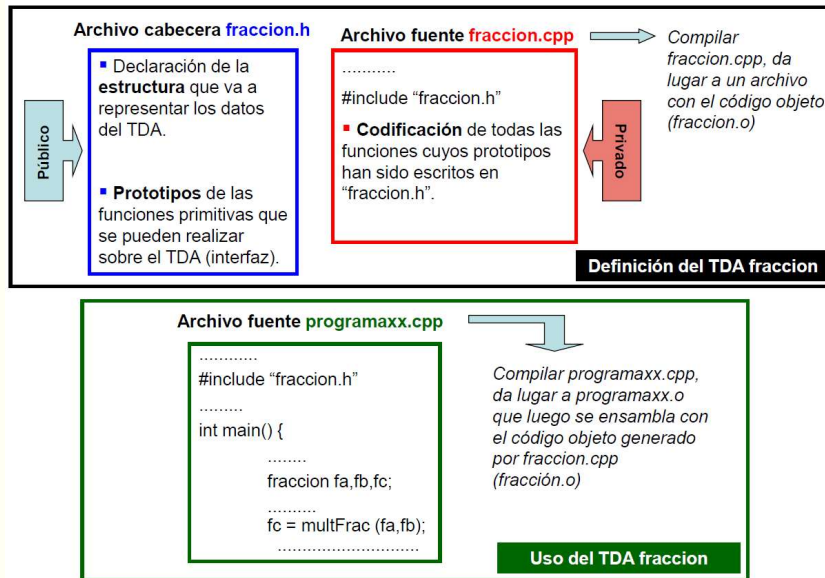
Estos archivos de cabecera se incluyen (mediante la directiva al preprocesador `#include`) en los archivos que contienen la codificación de las funciones, archivos fuente y también en los archivos de código que hagan referencia a algún elemento del archivo de inclusión.

Al implementar un TDA en C++, se agrupa en este archivo de inclusión o cabecera la representación de los datos y la interfaz del TDA (representada por los prototipos de las funciones).

Luego, en los archivos de programas que vayan a utilizar el TDA se debe escribir la directiva: `#include "tipoTDA.h"`

26

Definición y uso de TDA en C++



27

Algunas consideraciones de uso de TDA en C++

- Los programas fuente que vayan a utilizar un **TDA** necesitan conocer los datos públicos del mismo, es decir su **representación** y las **funciones primitivas** que se tienen disponibles, por ello se debe incluir en ellos el archivo de cabecera (.h).
- La implementación de las funciones primitivas es transparente para el programa que usa el TDA. Si bien debe existir y ensamblarse, el programador no necesita conocer ese código.
- Más aún, el archivo fuente (.cpp) que tiene el código de las funciones, puede cambiarse y solo será necesario recompilarlo, sin que esto afecte al programa que usa el TDA, siempre que la representación e interfaz se mantengan.
- Los compiladores de C++ disponen de herramientas para el manejo cómodo de aplicaciones modulares..

28

Otro ejemplo: Número complejo

- Otro tipo de dato numérico que puede ser necesario manejar son los números complejos, que no son tipos básicos (primitivos). Se puede generar un TDA en forma similar a las fracciones:

Archivo cabecera **complejo.h**

```
struct complejo{
    float preal;
    float pimag;
};

complejo sumComp (complejo a,complejo b);
complejo asigComp (float x,float y);
int complguales (complejo a, complejo b);
.....
```

Números complejos: describen la suma de un número real y un número imaginario

Archivo fuente **complejo.cpp**

```
.....
#include "complejo.h"

complejo sumComp (complejo a,complejo b)
{ complejo aux;
  aux.preal = a.preal + b.preal;
  aux.pimag = a.pimag + b.pimag;
  return aux;
}

complejo asigComp (float x,float y)
.....

int complguales (complejo a, complejo b)
{
  return( (a.preal == b.preal) && (a.pimag ==b.pimag));
}
.....
```

29

Otro ejemplo: Número complejo

- Luego, un programa que maneje números complejos puede ser:

Archivo fuente **programayy.cpp**

```
.....
#include "complejo.h"
.....
{ float r, i;
  complejo ca, cb, cc;
  cin >> r >> i ;
  ca = asigComp (r,i);
  cb = asigComp (3.0,5.5);
  cc = sumComp (ca,cb);
  cout <<"El resultado de la suma es : " << endl ;
  printComp (cc);
  system("pause");
}
```

Esta función imprime: cc.preal, "+", cc.pimag, " i"

30

Otro ejemplo: Un punto del plano

- Si debemos trabajar con puntos del plano, podemos también construir un TDA:

Archivo cabecera **puntop.h**

```
struct puntop
{ float abs;
  float ord;
};

puntop sumVect (puntop a,puntop b);
puntop asigPtop (float x,float y);
int ptoplguales (puntop a, puntop b);
int distancia (puntop a, puntop b);
.....
```

Archivo fuente **puntop.cpp**

```
.....
#include "puntop.h"

puntop sumVect (puntop a, puntop b)
{ puntop aux;
  aux.abs = a.abs + b.abs;
  aux.ord = a.ord + b.ord;
  return aux;
}

int ptoplguales (puntop a, puntop b)
{
  return ( a.abs == b.abs && (a.ord ==b.ord));
}

float distancia (puntop a, puntop b)
{
  return( sqrt( pow((a.abs - b.abs),2) +
    pow ((a.ord-b.ord),2) );
} .....
```

31

Otro ejemplo: Una recta en el plano

- Si debemos trabajar con rectas del plano, podemos considerar que una recta está caracterizada por tres coeficientes de su ecuación implícita ($ax+by+c=0$) y así construir un TDA:

Archivo cabecera **recta.h**

```
struct recta
{ float a; float b; float c;
};

recta asigRecta (float x,float y,float z);
int esVertical (recta r);
int esHorizon (recta r);
int sonParalelas (recta r, recta q);
int sonPerpend (recta r, recta q);
int sonOblicuas (recta r, recta q);
float pendiente (recta r);
float ordorigen (recta r);
float intersEjex (recta r);
float absIntersec (recta r, recta q);
float ordIntersec (recta r, recta q);
.....
```

Archivo fuente **recta.cpp**

```
.....
#include "recta.h"

int esVertical (recta r)
{ return ( r.b==0); }

int sonParalelas (recta r, recta q)
{ return( r.a*q.b == q.a * r.b) ; }

float ordorigen (recta r, recta q)
{ return (- r.c / r.b);}

float intersEjex (recta r)
{ return (- r.c / r.a);}

float absIntersec (recta r, recta q)
{ return( ((q.c/q.b)-(r.c/r.b)) / ((r.a/r.b)-(q.a/q.b)) );}
```

32

Otro ejemplo: Una recta en el plano

- Y un programa que use rectas:

Archivo fuente `programazz.cpp`

```
.....
#include "recta.h"

{ float m,n,o;
  recta r1,r2;

  cin >>m >>n >>o ;
  r1= asigRecta (m,n,o);
  if (! esVertical (r1))
    {cout <<"pendiente: " << pendiente(r1) ;};

  cin >> m >>n >> o;
  r2= asigRecta (m,n,o);
  if (! sonParalelas(r1,r2))
    {cout <<"el punto de interseccion de las rectas es : ( " << absIntersec (r1, r2) << " , "
      << ordIntersec (r1,r2) << " " );};

  system("pause");
}
```

33

El rol de la abstracción

- La abstracción ha sido clave en la Programación, como mecanismo para poder controlar problemas de mayor complejidad ("los humanos hemos desarrollado una técnica excepcionalmente potente para tratar la complejidad: abstraernos de ella")
- Ante la dificultad de dominar en su totalidad los objetos complejos, se ignoran los detalles no esenciales, tratando en su lugar con el modelo ideal del objeto y centrándonos en el estudio de sus aspectos esenciales
- La abstracción es la capacidad para encapsular y aislar información del diseño y ejecución (ocultamiento de información)

En la historia del software la abstracción ha sido clave:

- Nombres mnemotécnicos para las instrucciones, en lugar de representación binaria
- Macroinstrucciones
- Estructuras de control
- Procedimientos y funciones
- TDA = Representación (datos) + Operaciones (funciones y procedimientos).

34

Ventajas de los TDA

- Permite una mejor conceptualización y modelización del mundo real.
- Mejora la representación y la comprensibilidad. Clarifica los objetos basados en estructuras y comportamientos comunes.
- Mejora la robustez del sistema. Los TDA permiten la comprobación de tipos para evitar errores en tiempo de ejecución.
- Mejora el rendimiento (optimiza tiempos de compilación).
- Separa la implementación de la especificación. Permite la modificación y mejora de la implementación sin afectar la interfaz pública del TDA.
- Permite la extensibilidad del sistema. Los componentes de software reusables son más fáciles de crear y mantener.

35

Ejemplo de un TDA Estudiante

```
// Archivo estudiante.h
// Definimos un struct
struct Estudiante {
    int legajo;
    string nombre;
    float promedio;
};
// Prototipos de funciones asociadas
void cargarEstudiante(Estudiante &e);
void mostrarEstudiante(const Estudiante &e);
```

36

Ejemplo de un TDA Estudiante

```
// Archivo estudiante.cpp

#include <iostream>
#include <cstring>
#include "estudiante.h"
using namespace std;

// Implementación de funciones
void cargarEstudiante(Estudiante &e) {
    cout << "Ingrese legajo: ";
    cin >> e.legajo;
    cin.ignore(); // limpiar buffer
    cout << "Ingrese nombre: ";
    cin.getline(e.nombre, 40);
    cout << "Ingrese promedio: ";
    cin >> e.promedio;
}

void mostrarEstudiante(const Estudiante &e) {
    cout << "\n--- Datos del estudiante ---\n";
    cout << "Legajo: " << e.legajo << endl;
    cout << "Nombre: " << e.nombre << endl;
    cout << "Promedio: " << e.promedio << endl;
}
```

37

Ejemplo de un TDA Estudiante

```
// Archivo main.cpp

#include <iostream>
#include "estudiante.h" // incluimos nuestro header
using namespace std;

int main() {
    Estudiante alumno;

    cargarEstudiante(alumno);
    mostrarEstudiante(alumno);

    return 0;
}
```

38

Implementar el TDA - Practica

Círculo:

- Archivo de interfaz
- Definir la estructura Circulo
- Circulo crear_circulo(...);
- float perimetro(Circulo c);
- float area(Circulo c);

39

Algoritmos y Estructuras de Datos – AEDD – Agenda 11/09

- ✓ Definición de Estructura.
- ✓ Definición de dato TDA.
- ✓ Tipo de dato: Fracción.
- ✓ Implementación y uso de TDA.
- ✓ Definición y uso de TDA.
- ✓ Ejemplo: Punto en el plano.
- ✓ Ejemplo: Recta en el plano.
- ✓ Rol de la abstracción.
- ✓ Ventajas de los TDA

¡¡Objetivo cumplido!!!

40

AEDD – Otras tareas !!!!!

“Abstracción en lenguajes de programación”,
pág. 21

“Tipos abstractos de datos”, pág. 23

Libro: “Estructura de datos en C++” – Luis
Joyanes Aguilar

41



**ALGORITMOS Y
ESTRUCTURA DE
DATOS**

Comisión F – 2026 – Cursado Anual
¿DUDAS?

42

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¡Muy buena semana!

